Ampliación de Estructura de Computadores

DRAFT

DRAFT

Copyright © 2019 Juan Marín Noguera, juan.marinn@um.es.

Esta obra es libre: puedes distribuirla y/o modificarla bajo los términos de la licencia Creative Commons Attribution-ShareAlike 3.0. Para ver una copia de esta licencia, visita https://creativecommons.org/licenses/by-sa/3.0/legalcode. Esta obra viene sin ninguna garantía, al máximo alcance permitido por la legislación aplicable.

Gracias por comprar este ejemplar. Al hacerlo estás ayudando a los refugiados de Atenas, pues los beneficios se destinan íntegramente a Project Elea, un grupo de voluntarios que trabajan con los residentes del campo de refugiados de Eleonas para mejorar los estándares de vida y el bienestar comunitario dentro de este. Ver https://projectelea.org/about-us/ para más información.

En el muy improbable caso de que el proyecto deje de recibir donaciones antes de enviar el dinero, este se destinaría a otra ONG.

Capítulo 1

Análisis de prestaciones

Llamamos **rendimiento** de una CPU al realizar una cierta tarea al inverso del tiempo medio que tarda en realizarla. Un ordenador es más rápido que otro si disminuye el tiempo de respuesta, por ejemplo, con un ciclo de reloj más corto, o aumenta la productividad, por ejemplo, trabajando en paralelo. Decimos que X es un n % más rápido que Y si

$$\frac{\text{Rendimiento } X}{\text{Rendimiento } Y} = 1 + \frac{n}{100}$$

A nivel de hardware, podemos medir la velocidad en ciclos o bytes por segundo:

$$T_{\text{CPU}} = \text{instrucciones} \cdot {}^{\text{ciclos}}/_{\text{instrucción}} \cdot {}^{t}/_{\text{ciclo}}.$$

Llamamos CPI a la media de ciclos por instrucción. Si la CPU tiene n instrucciones con una media de CPI_k ciclos por instrucción de tipo k, que se ejecuta con frecuencia f_k, entonces

$$\text{CPI} = \sum_{k=1}^{n} \text{CPI}_k f_k.$$

También podemos medir las instrucciones por unidad de tiempo. La medida en MIPS, millones de instrucciones por segundo, es fácil de comprender pero dependiente del conjunto de instrucciones y del programa; por ejemplo, la frecuencia de instrucciones en coma flotante, normalmente más lentas, es muy variable.

Medir en MFLOPS, millones de instrucciones en coma flotante por segundo, soluciona este problema, pero no aplica a todos los programa y, ni todas las máquinas tienen las mismas instrucciones de coma flotante, ni todas tardan lo mismo.

La **ley de Amdahl** afirma que, si una mejora hace que una parte de la ejecución que se ejecutaba en una parte f del tiempo total se ejecute S veces más rápido, el aumento de la velocidad global es

$$\frac{1}{(1-f) + \frac{f}{S}}.$$

De aquí que debemos centrarnos en optimizar el caso más frecuente. Este suele ser el más fácil de mejorar.

DRAFT

Si conocemos el rendimiento de distintos programas (bajo ciertos parámetros) en distintas máquinas, podemos comparar mediante la media aritmética de los tiempos de ejecución ponderada respecto a la frecuencia con la que se ejecutan los programas, o con la **media armónica ponderada** de las velocidades,

$$\overline{v} := \frac{1}{\sum_k \frac{f_k}{v_k}}.$$

Si los programas son muy distintos, se suele hacer la media geométrica de la razón del tiempo ejecución del programa en la máquina que medimos respecto al tiempo en una máquina de referencia. Esto no predice los tiempos de ejecución, pero da una medida bastante coherente.

El rendimiento se mide respecto a **programas de prueba**, de distintos tipos de mayor a menor fiabilidad:

- Programas reales, a veces simplificados.
- **Núcleos** o *kernels*, partes de programas reales, como Livermore Loops y Linpack.
- ***Benchmarks* sintéticos**, parecidos a los núcleos, como Whetstone y Dhrystone.
- ***Benchmarks* reducidos**, pequeños y fácilmente portables, como QuickSort o Puzzle.

Su uso puede llevar a conclusiones incorrectas, y es importante que las pruebas sean reproducibles indicando parámetros como las versión del programa, la del sistema operativo, la del compilador, el nivel de optimización, la entrada, la memoria principal, el número y tipo de discos o la CPU concreta.

SPEC (*System Performance Eval. Coop.*) es una serie de programas de pruebas para el rendimiento de CPU, gráficos, E/S, bases de datos, etc. En 1989, la parte para CPU constaba de 10 programas que producían un único número. En 1992, esto evolucionó a SPECInt92, 6 programas con enteros, y SPECfp92, con 14 para coma flotante. En 1995 hay 8 de enteros, SPECint95, y 10 de coma flotante, SPECfp95, y se fijan los parámetros de compilación en SPECint_base95 y SPECfp_base95. En el 2000 hay 12 programas con enteros, SPECint2000, y 14 con coma flotante, SPECfp2000. En 2006 hay 12 de enteros, SPECint2006, y 17 en coma flotante, en SPECfp2006. En 2017 hay 43 programas organizados en 4 suites.

TPC (*Transaction Processing Council*) es una serie de programas de prueba para la capacidad de manejo de transacciones: accesos y actualizaciones de bases de datos. El rendimiento se mide en transacciones por segundo con un requisito de tiempo de respuesta, y combina CPU, E/S, red, sistema operativo, gestor de bases de datos, etc. En 1985 se creó **TPC-A**, y actualmente hay varias suites que simulan distintos entornos:

- **TPC-C**: Entorno complejo de transacciones en línea para el manejo de pedidos.
- **TPC-H**: Entornos de toma de decisiones.
- **TPC-W**: Actividades de comercio electrónico.
- **TPC-App**: Actividades de un servidor de aplicaciones y servicios web.

DRAFT

Capítulo 2

Segmentación básica

La **segmentación** consiste en dividir el cauce de ejecución (*pipeline*) en **etapas** o **segmentos**, conectados uno al siguiente, de forma que las instrucciones entran al principio del cauce, pasan por las etapas y salen por el final, pudiendo haber hasta una instrucción en cada etapa, aumentando por tanto la **productividad** (*throughput*) al aumentar la frecuencia de reloj o disminuir el número de ciclos por instrucción. El **ciclo máquina** es el tiempo necesario para asegurarnos de que las instrucciones han terminado una etapa y pueden pasar a la siguiente.

Idealmente la aceleración al segmentar es igual al número de etapas del cauce, pero en general estas etapas no están perfectamente balanceadas y suele haber un coste adicional por la segmentación, con lo que las instrucciones tardan más en ejecutarse a cambio de poder hacerlo varias a la vez. El que el tiempo de ejecución de cada instrucción no disminuya pone una cota práctica al número de etapas.

DRAFT

2.1. DLX

Es una arquitectura RISC abierta parecida a MIPS. Registros:

- 32 de propósito general de 32 bits, `R0`, ..., `R31`. `R0` vale 0 e ignora las escrituras.
- 32 de punto flotante de 32 bits, `F0`, ..., `F31`, que se pueden usar como de simple precisión o, juntando uno con índice par con el siguiente, como de doble precisión.
- De estado de punto flotante (`fp`), contador de programa (`PC`) y de direcciones de interrupción (`IAR`).

Soporta enteros de 8, 16 y 32 bits, y la memoria es direccionable por bytes con direcciones de 32 bits.

2.1.1. Formatos de instrucción

Con inmediato

31 – 26	25 – 21	20 – 16	15 – 0
Operación	Fuente	Destino	Inmediato con signo

OP Rd , Imm(Rs), *OP Imm(Rs) , Rd*, *OP Rd , Rs , #Imm* u *OP Rs , tag*.

- Carga: LB, LBU, LH, LHU, LW, LF, LD. Se accede a la dirección «Fuente + Inmediato».
- Almacenamiento: SB, SH, SW, SF, SD.
- ALU con inmediato: ADDI, ADDUI, SUBI, SUBUI, ANDI, ORI, XORI, LHI, SLLI, SRLI, SRAI, SLTI, SGTI, SLEI, SGEI, SEQI, SNEI.
- Salto condicional: BEQZ, BNEZ, BFPT, BFPF. *OP Rs , tag*. Se salta a «Sig. instr.+Inmediato» si «Fuente» cumple la condición.
- Salto respecto a registro: JR, JALR.

Entre registros

31 – 26	25 – 21	20 – 16	15 – 11	10 – 0
Operación	Fuente 1	Fuente 2	Destino	Función

OPFN Rd , Rs1 , Rs2.

- Transferencias entre registros: MOVI2S, MOVS2I, MOVF, MOVD, MOVFP2I, MOVI2FP.
- ALU registro-registro: ADD, ADDU, SUB, SUBU, MULT, MULTU, DIV, DIVU, AND, OR, XOR, SLL, SRL, SRA, SLT, SGT, SLE, SGE, SEQ, SNE.
- Punto flotante: ADDF, SUBF, MULTF, DIVF, LTF, GTF, LEF, GEF, EQF, NEF, ADDD, SUBD, MULTD, DIVD, LTD, GTD, LED, GED, EQD, NED, CVTI2F, CVTI2D, CVTF2I, CVTF2D, CVTD2I, CVTD2F.

DRAFT

De salto

31 – 26	25 – 0
Operación	Desplazamiento respecto a siguiente instrucción

OP tag.

- Bifurcación: J, JAL.
- Excepción y retorno de excepción: TRAP, RFE.

2.1.2. Implementación sin segmentación

Implementamos un subconjunto de DLX formado por las instrucciones de carga y almacenamiento de palabras, saltos condicionales y operaciones enteras de la ALU, en 5 etapas:

1. Búsqueda de instrucción (**IF**, *Instruction Fetch*).

   ```
   IR <- Mem[PC], NPC <- PC + 4
   ```

2. Decodificación de la instrucción (**ID**, *Instruction Decode*).

   ```
   A <- Regs[IR[25..21]], B <- Regs[IR[20..16]],
   Imm <- (IR[16]*16++IR[16..31])
   ```

3. Ejecución (**Ex**).

   a) Referencia a memoria: `ALUOut <- A + Imm`

   b) ALU registro-registro: `ALUOut <- A func B`

   c) ALU con inmediato: `ALUOut <- A op Imm`

   d) Salto: `ALUOut <- NPC + Imm, Cond <- A op 0`

4. Acceso a memoria (**Mem**).

   a) Carga: `LMD <- Mem[ALUOut]`

   b) Almacenamiento: `Mem[ALUOut] <- B`

   c) Salto: `PC <- if Cond { ALUOut } else { NPC }`

5. Pos-escritura (**WB**, *Write-Back*).

   a) ALU registro-registro: `Regs[IR[15..11]] <- ALUOut`

   b) ALU registro-inmediato: `Regs[IR[20..16]] <- ALUOut`

   c) Carga: `Regs[IR[20..16]] <- LMD`

2.1.3. Implementación segmentada

Entre una etapa y la siguiente, añadimos registros, los ***pipeline registers*** o ***pipeline latches***, que guardan los datos generados por la primera que son necesarios más adelante, y que suelen llamarse como los nombres de las etapas que separan, por ejemplo, «IF/ID». Modificamos el cauce para que el cálculo de la dirección siguiente, en vez de realizarse en (4c), se realice en (1), con conexiones «hacia atrás».

Como dos instrucciones distintas no pueden usar un mismo recurso al mismo tiempo, usamos cachés separadas para instrucciones (usada en (1)) y datos (usada en (4)). El banco de registros se lee en (2) y se escribe en (5). Si bien estas operaciones se pueden realizar a la vez si se trata de registros distintos, esto puede resultar en que se lea el valor de un registro antes de que lo escriba una operación anterior. Se añaden también **señales de control**.

DRAFT

2.2. Riesgos

Idealmente el CPI sería 1, pero existen **riesgos** o *hazards*, situaciones en que una instrucción no puede avanzar a la siguiente etapa de forma segura.

Pueden ser resueltos, entre otras formas:

- Por hardware, detectando la situación y parando la instrucción problemática, cuyo CPI aumenta tanto como ciclos dure la detención, y las siguientes, mientras avanzan las anteriores.

- Por software, añadiendo instrucciones NOP (*No OPeration*) para separar las instrucciones dependientes. Esto es más sencillo a nivel de hardware, pero complica el compilador e impide la portabilidad del código máquina.

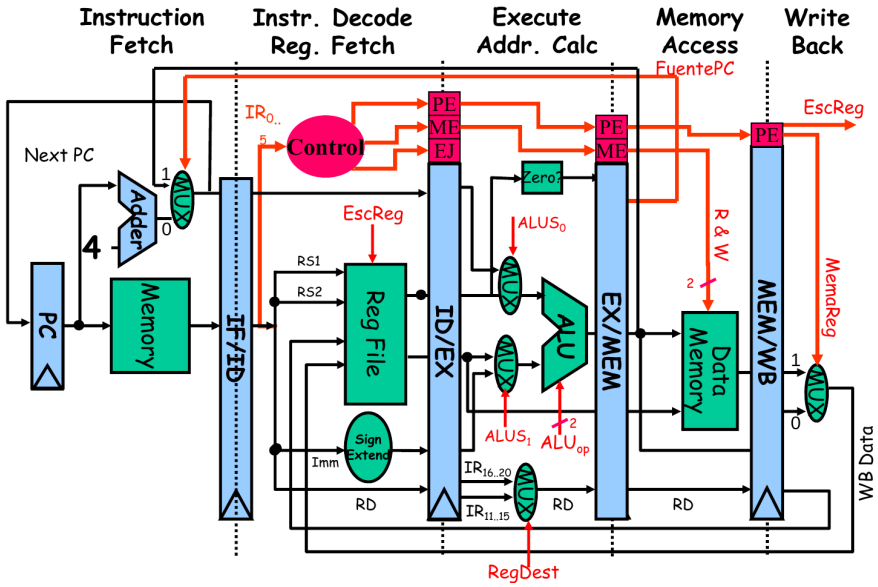

Figura 2.1: Esquema de CPU segmentada con señales de control.

DRAFT

2.2.1. Riesgos estructurales

Surgen cuando varias instrucciones necesitan usar el mismo recurso para cosas distintas pero este no se ha replicado lo suficiente o no se ha segmentado. Por ejemplo, si en un banco de registros con un puerto de lectura, dos instrucciones necesitan escribir a la vez; si una instrucción requiere acceder a memoria pero hay una única memoria para datos e instrucciones (lo que se puede solucionar con cachés de primer nivel separadas), o si una unidad funcional necesita varios ciclos para hacer una operación pero se usa en dos instrucciones consecutivas.

Además de las soluciones anteriores, otra solución puede ser dedicar más hardware, pero esto puede no merecer la pena si el coste es elevado o en caso de unidades funcionales que no se usan mucho.

Si se quiere detectar la situación para pausar la ejecución, debe ser sencillo, a partir del código de operación, detectar los recursos que usa la instrucción.

2.2.2. Dependencias

- **De datos** o **verdaderas**: Una instrucción usa un resultado producido por otra anterior.
- **De nombre**
  - **Antidependencias**: Una instrucción usa el valor de un registro o posición de memoria que algo después es sobrescrito.
  - **Dependencias de salida**: Varias instrucciones escriben en el mismo registro o posición de memoria.
- **De control**: La ejecución de una instrucción depende de si se toma un salto o no.

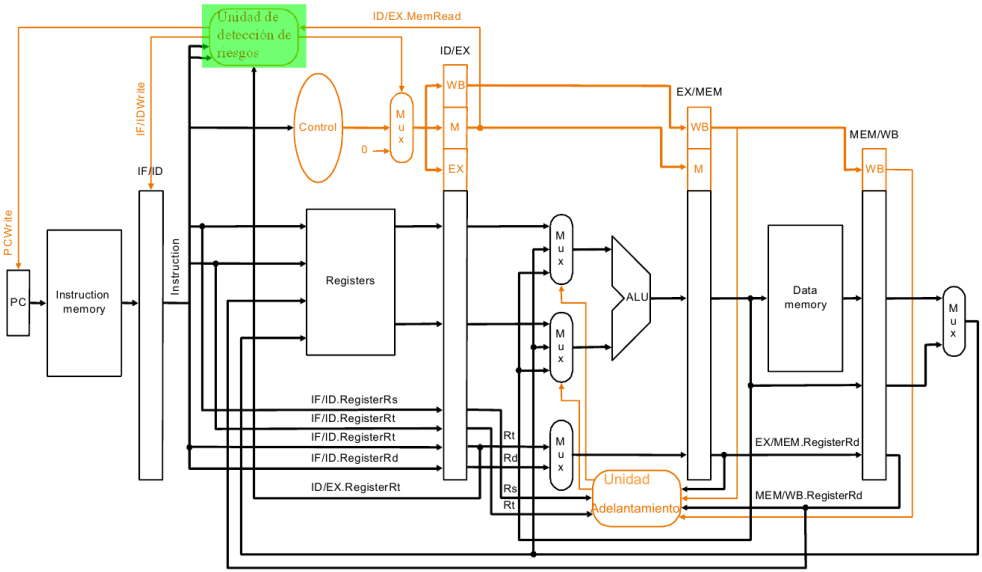

Figura 2.2: Esquema del mecanismo de detección de riesgos de datos y parada.

DRAFT

2.2.3. Riesgos de datos

- **RAW** (*Read After Write*): Si hay una dependencia de datos, una instrucción podría leer un operando antes de que otra anterior lo escriba.
- **WAR** (*Write After Read*): Si hay una antidependencia, una instrucción podría escribir un antes de que otra anterior lo lea. Un compilador las puede evitar renombrando registros.
- **WAW** (*Write After Write*): Si hay una dependencia de salida, una instrucción puede escribir un operando antes de que otra anterior lo haga. Solo es posible si varias etapas pueden escribir en el mismo sitio o si se permite que otra instrucción posterior continúe mientras la anterior está detenida.

En nuestra implementación de DLX, solo hay una fase de acceso a memoria y es bloqueante, por lo que todos los riesgos de datos se dan en los registros. La única etapa de escritura a registros va siempre después de la única de lectura, por lo que no puede haber riesgos WAR o WAW, pero sí riesgos RAW.

2.2.4. Riesgos de control

Producidos por dependencias de control. En nuestra implementación, PC no cambia hasta el final de Mem, por lo que una instrucción de salto supone 3 ciclos de parada, latencia que puede reducirse averiguando antes si un salto se toma o no y calculando antes la dirección efectiva del salto. Para ello, podemos mover la unidad detectora de ceros a ID y calculamos la dirección de salto sea cual sea el resultado de la comparación, añadiendo para ello un sumador, con lo que podemos tomar la decisión del salto en ID y solo tendríamos un ciclo de parada.